Week 2 – Advanced Concepts and Practical Implementation

Overview

Week 2 focused on advanced Java concepts including Generics, Exception Handling, Collections Framework, File Handling, and JSON processing. The goal was to strengthen problem-solving skills by implementing real-world scenarios such as dynamic containers, custom exception systems, sorting mechanisms, log analysis, and JSON-based data processing.

Implementation Details

1. Generics – Generic Container

- Created a generic class GenericContainer<T> using arrays only.
- Implemented dynamic resizing logic.
- Methods added: add(), get(), size(), printAll().
- Implemented static generic methods for numeric types:
  - sum()
  - average()
- Tested with:
  - String To-Do list
  - Integer/Double score container

2. Exception Handling – Library Management System

- Created Book class with required attributes.
- Implemented custom exceptions for:
  - Invalid Book ID
  - Invalid Price
  - Book Not Available
- Used throws keyword for validation method.
- Stored validated books in ArrayList.

- Sorted books:
  - Title (descending)
  - Price (ascending if titles match)
- Handled all exceptions in main() using try-catch.
- Also implemented division method using throws and handled ArithmeticException.

3. Collections Framework

- Used Set for removing duplicates and printing elements in descending order.
- Implemented Comparable for sorting students by grade (highest first).
- Used Comparator for alternate sorting by name.
- Wrote pseudo-code for Singly Linked List:
  - Node structure
  - Insert logic
  - Traversal logic

4. File Handling – Log Analyzer

- Read log file manually without external libraries.
- Extracted only ERROR logs.
- Generated report containing:
  - Total ERROR count
  - Request identifiers
  - Project package stack trace lines (com.somebank.aem.platform)
  - File name + line number extraction
  - Highlighted repeated occurrences with frequency and associated request IDs

5. JSON Processing

- Parsed JSON using JSONObject.
- Extracted:
  - Dates
  - OrderId
  - CustomerName

- Created POJO classes and deserialized using Gson.
- Implemented:
    - Total order amount calculation
    - Payment status display
    - Failed orderIds list
    - Total revenue calculation
    - Highest order value detection
    - Revenue per date
    - Highest revenue date
    - Orders by input date

Testing & Results

- Generic container resized dynamically without data loss.
- Custom exceptions triggered correctly for invalid inputs.
- Sorting logic worked as per defined conditions.
- Log analyzer correctly filtered and reported ERROR entries.
- JSON processing produced accurate revenue and order analysis outputs.

Challenges & Solutions

- Dynamic array resizing logic - implemented manual array copy mechanism.
- Custom exception design -clarified checked vs unchecked exceptions.
- Sorting confusion (Comparable vs Comparator) - tested both independently.
- Log parsing complexity - used string splitting and pattern matching carefully.
- JSON revenue calculation - ensured correct numeric conversion.

Next Steps

- Practice multithreading concepts.
- Work on stream API and lambda expressions.
- Improve exception design patterns.
- Implement mini-project combining file handling + collections + JSON.